AI-Enhanced Multi-Agent Inventory Reconciliation System

Prompt Engineering Document

Prompt 1: Understanding the Problem and Designing the Solution

I want to create an AI-powered Inventory Reconciliation Application that enables users to reconcile inventory records with actual infrastructure assets.

The primary objective is to reduce the manual audit process and identify:

- Missing Assets
- Untracked Assets
- Configuration Mismatches
- Naming Mismatches

The application should also validate uploaded files, perform reconciliation, generate AI-based analysis, recommendations, executive summaries, and reports.

Propose a complete system architecture, workflow, database design, API structure, and technology stack for this solution.

Prompt 2: Multi-Agent Workflow Design

Support me in designing a multi-agent architecture for an Inventory Reconciliation Platform.

The system should contain specialized agents responsible for:

- Inventory Validation
- Reconciliation
- AI Analysis
- Recommendation Generation
- Executive Summary Generation
- AI Chat Assistance

Describe how each agent works, what input it receives, what output it produces, and how information is exchanged between agents.

Prompt 3: Backend Development

Develop a FastAPI backend for the Inventory Reconciliation Platform.

The backend should:

- Use FastAPI and PostgreSQL
- Follow a modular folder structure
- Use SQLAlchemy for database operations
- Implement JWT Authentication
- Support Role-Based Access Control (RBAC)

Provide REST APIs for:

- Authentication
- File Upload
- Reconciliation
- Reporting
- Chatbot Services

Include proper validation, logging, and error handling.

Prompt 4: Building the Inventory Validation Agent

Create an intelligent Inventory Validation Agent.

The agent should:

- Read CSV inventory files
- Read JSON infrastructure files
- Detect missing values
- Detect duplicate records
- Detect invalid records
- Validate file structure

Return a structured validation report.

Prompt 5: Building the Reconciliation Agent

Create a Reconciliation Agent.

The agent should compare:

- Inventory.csv
- Live Inventory.json

Identify:

- Missing Assets
- Untracked Assets
- Configuration Mismatches
- Naming Mismatches

Use Pandas and RapidFuzz for accurate matching and reconciliation.

Return structured reconciliation results.

Prompt 6: Building the AI Analysis Agent

Create an AI Analysis Agent using Gemini 2.5 Flash.

The agent should:

- Analyze discrepancies
- Perform root cause analysis
- Assess risk levels
- Explain business impact

Generate professional analysis reports to support operational decision-making.

Prompt 7: Building the Recommendation Agent

Create a Recommendation Agent.

The agent should:

- Suggest corrective actions
- Recommend verification steps
- Prioritize identified issues

- Suggest long-term prevention strategies

Provide actionable and business-oriented recommendations.

Prompt 8: Report Generation Agent

Create a Reporting Agent.

The agent should generate:

- Reconciliation Reports
- Audit Reports
- Executive Reports
- Risk Assessment Reports

Reports should be exportable as professional PDF documents.

Prompt 9: Frontend Development

Develop a modern frontend application using React and Tailwind CSS.

The application should include:

- Landing Page
- Login Page
- Dashboard
- Upload Page
- Results Page
- Reports Page
- AI Chatbot

The user interface should be responsive, professional, and suitable for enterprise environments.

Prompt 10: AI Chatbot Assistant

Create an AI-powered Inventory Intelligence Assistant using Gemini.

The assistant should answer questions related to:

- Missing Assets
- Inventory Health
- Risk Analysis
- Reports
- Executive Summaries

Provide a chat-based interface integrated with backend APIs.

Prompt 11: Final Integration

Integrate the frontend, backend, database, AI agents, Gemini API, and LangGraph into a single application.

Ensure that:

- All agents communicate through a clear orchestration pipeline
- Reconciliation executes successfully
- Reports and executive summaries are generated correctly
- Frontend and backend communication functions seamlessly

Prepare the application for demonstration, testing, and deployment.

Development Approach

The project was developed using a Prompt-Driven Engineering methodology.

Implementation Phases

1. Requirement Analysis
2. System Architecture Design
3. Multi-Agent Workflow Design
4. Backend Development
5. Database Design
6. Inventory Validation Development
7. Reconciliation Engine Development
8. AI Analysis Development

9. Report Generation
10. Frontend Development
11. AI Integration
12. End-to-End Testing

Generative AI was used as a development accelerator, while architecture design, workflow planning, feature selection, system integration, testing, and project customization were carried out throughout the development lifecycle.